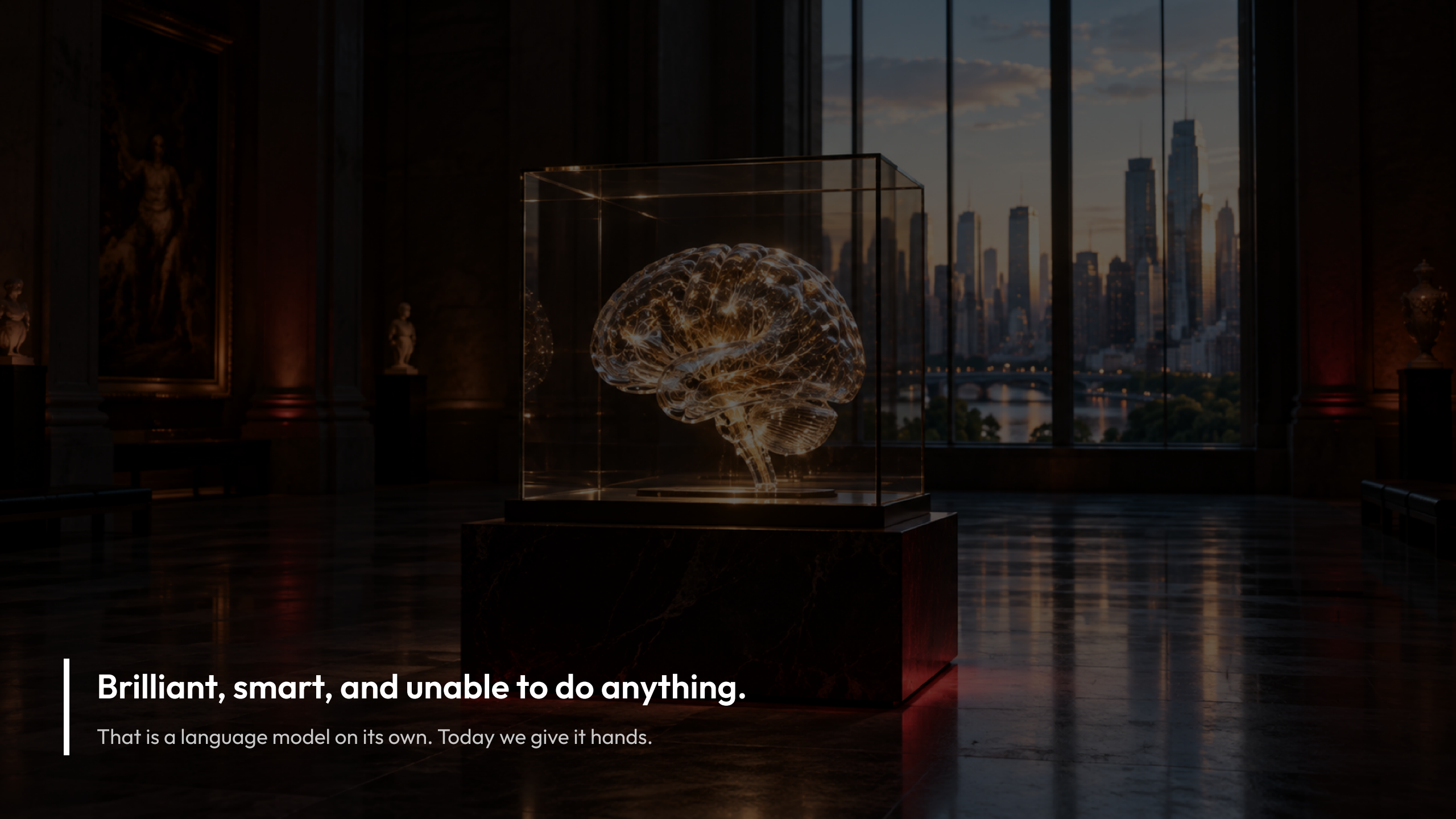
Agent Development Basics

LLM-Agents

Presented by: **Aleksandr Algazinov**

HSE University



Brilliant, smart, and unable to do anything.

That is a language model on its own. Today we give it hands.

What we mean by agent today

OUR DEFINITION · LECTURE 04

Anything beyond a bare model call is an agent: a system prompt, a tool, a loop, several agents.

A tool is optional.

THE COMMON DEFINITION

"An LLM agent runs tools in a loop to achieve a goal."

— Simon Willison, 2025

TODAY: THE COMMON ONE

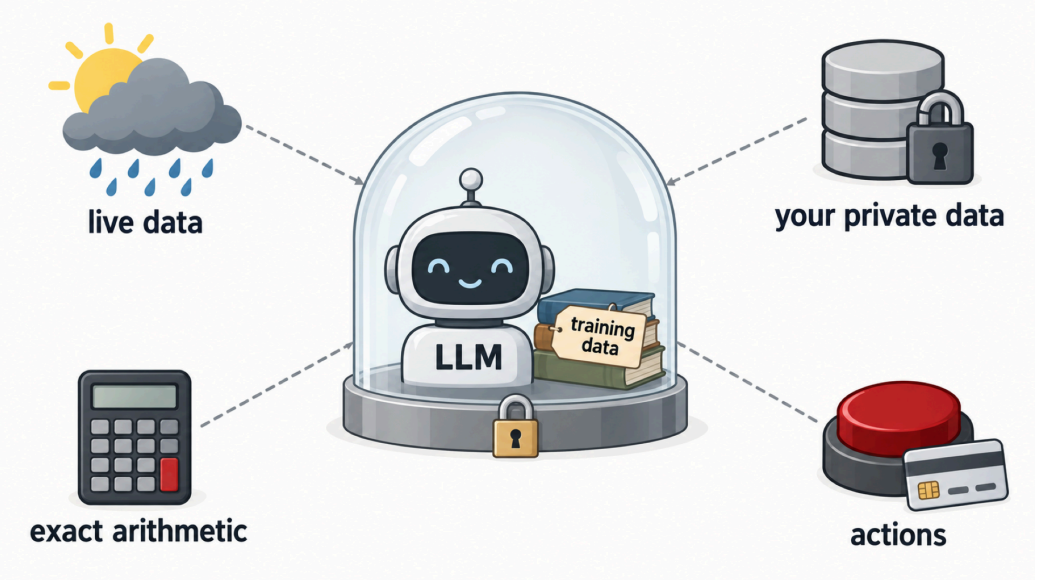
Tools and the loop are exactly what this lecture builds — on raw provider SDKs, so you see the wire format before Lecture 09 hides it. Neither definition is wrong; that argument was last week's.

Motivation

- **Live data** — *"Umbrella in Berlin right now?"*
- **Private data** — *"Where is my order #4471?"*
- **Exact computation** — *"How many working days until 3 November?"*
- **Acting** — *"Refund my double charge."*

WHAT FIXES IT

Not a better prompt. The weights were frozen at training time, and text is not an action. Each one needs a **tool**.



Ask it anyway

```
r = client.chat.completions.create(model=MODEL, messages=[
        {"role": "user", "content": "What's the weather in Berlin right now?"}])
print(r.choices[0].message.content)
```

```
I don't have access to real-time weather data, so I can't tell you the exact
conditions in Berlin right now. For the most up-to-date forecast, you could check
a weather app or site like weather.com, AccuWeather, or your phone's built-in
weather app. [...]
```

THE GOOD OUTCOME

It says it does not know — today's model, run on 2026-10-08.

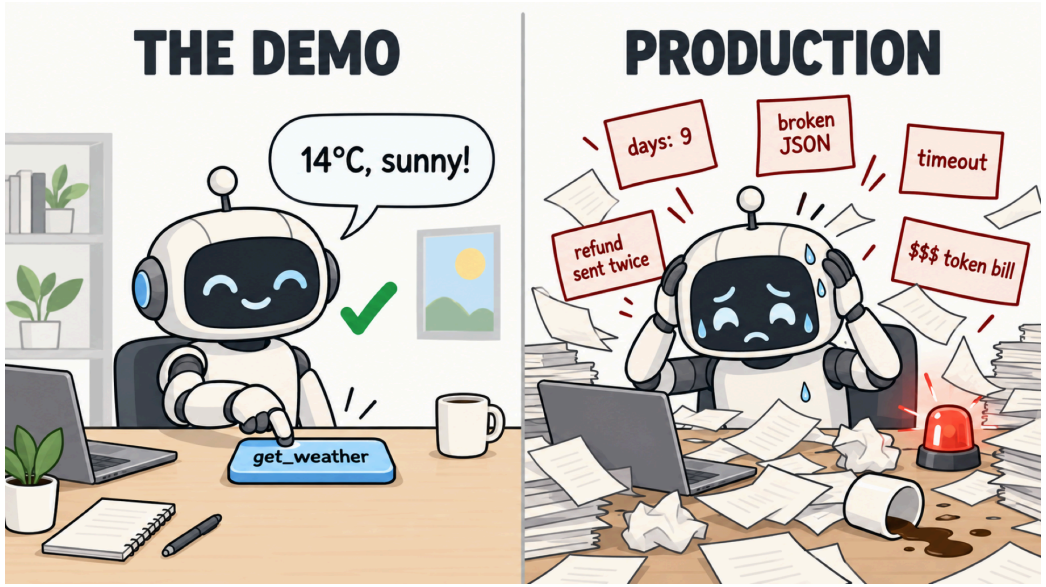
THE BAD ONE

A confident, invented temperature. Either way, the fix is a tool.

Once the model can act, what breaks

The demo works first time. Today is about the rest:

- Bad arguments → **Function Calling**
- Unparseable output → **Structured Output**
- A timeout on a refund → **Idempotency keys**
- A bill nobody explains → **Instrumentation**



ALREADY DOING IT

Klarna's assistant: 2.3M chats in month one (company-reported, 2024), refunds included — a side effect.

Setup — the key comes from the environment

Pinned: `openai==3.14.1` · `pydantic==2.13.5` · `python-dotenv==1.2.3` · `requests==2.34.2`

```
# openai==3.14.1 · python-dotenv==1.2.3 · run 2026-10-08
import os
from dotenv import load_dotenv
from openai import OpenAI
load_dotenv()                                  # API_KEY, BASE_URL, MODEL_NAME
MODEL = os.environ["MODEL_NAME"]
client = OpenAI(api_key=os.environ["API_KEY"],    # read at runtime,
                base_url=os.environ["BASE_URL"])  # never a literal
r = client.chat.completions.create(model=MODEL, messages=[
        {"role": "user", "content": "Reply with the single word: ready"}])
print(MODEL, "->", r.choices[0].message.content.strip())
```

```
dots-studio/dots-3-note-preview:free -> ready
```

One rule: the key is read at runtime, never written into a file git can see — Lecture 01's `.env`. Set `MODEL_NAME` to a model that lists `tools`, `structured_outputs` and `response_format`.

Tools

How an agent touches the world

What a tool is, to the model

THREE THINGS

- A **name**
- A **description**
- A typed **parameter schema**

THE DESCRIPTION IS NOT DOCUMENTATION

It is the only thing the model uses to decide whether to call this tool. Write it for the model, not for your teammates.

AND NOTHING ELSE

The model cannot see your implementation, your database, or your error handling. Only those three fields.

A taxonomy worth having

The categories carry different risk — and that decides how you handle a failure.

different categories carry different risk —
so each one handles failure differently

What makes a good tool boundary

DO

- One job per tool
- A name the model could guess
- Errors that tell the model what to do differently

AVOID

- A `do_everything(action, payload)` tool
- Errors that say only `"failed"`
- Twenty tools where five would do — every schema costs tokens, as we will measure

AN ERROR THAT TELLS THE MODEL WHAT TO DO DIFFERENTLY

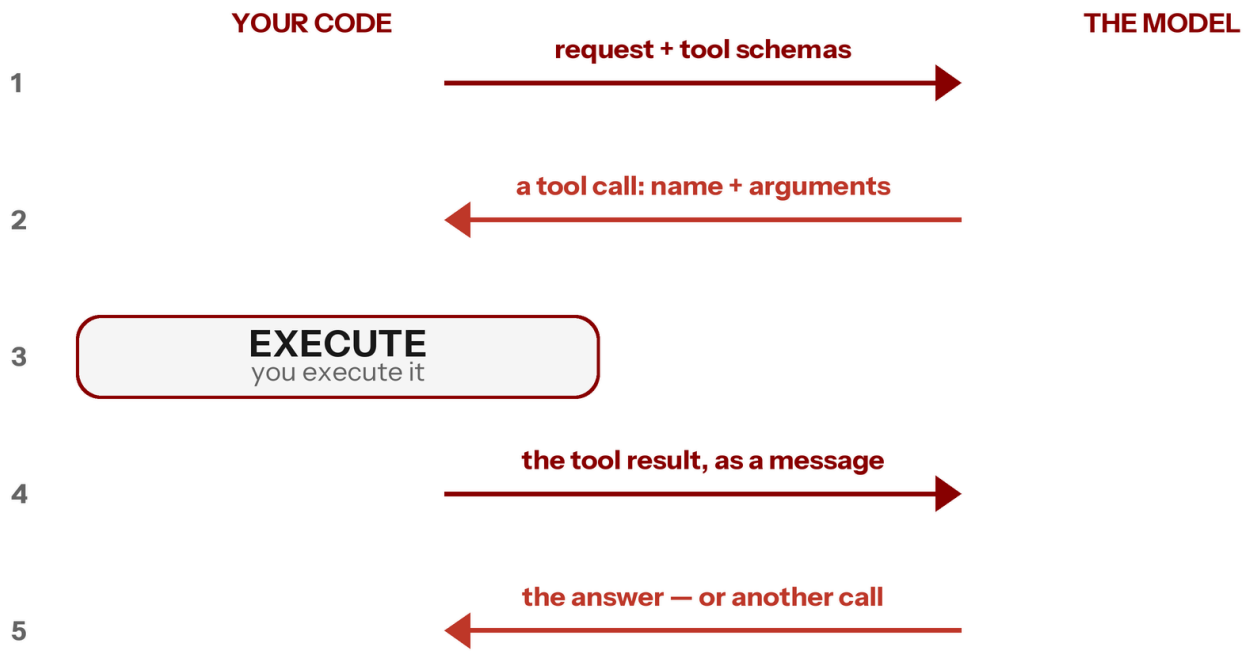
`{"error": "unknown tool; available: ['get_weather']"}` — not a stack trace, but the list it should have chosen from. You will see this one come back from a real run shortly.

Function Calling

Making last week's loop correct

The whole mechanism

Last week's ~14-line `run()` already made this round trip — with every parameter typed `string`, nothing validated, and `while True`. Today: the same loop, made **correct**.



the model only ever emits text — "calling a tool" is your code recognising a structured request and acting on it

A real schema — typed, and described per parameter

```python
from typing import Literal
from pydantic import BaseModel, Field
class WeatherArgs(BaseModel):
    city: str = Field(description="One city, in English, e.g. 'Berlin'")
    units: Literal["celsius", "fahrenheit"] = Field("celsius",
                description="Temperature units")
    days: int = Field(0, ge=0, le=3, description="0 = now, 1–3 = ahead")
WEATHER_TOOL = {"type": "function", "function": {
    "name": "get_weather",
    "description": "Current weather or a short forecast for ONE city. Use "
                   "when asked about conditions, temperature or forecast.",
    "parameters": WeatherArgs.model_json_schema(),   # what the model sees
}}
```

WHAT REACHES THE WIRE

`units` becomes an `enum`, `days` gets `minimum` / `maximum`.

ONE CLASS, TWO JOBS

Makes the schema here; validates the arguments two slides on.

`get_weather` — a real API, no key

```python
import requests
GEO, API = ("https://geocoding-api.open-meteo.com/v1/search",  # free,
            "https://api.open-meteo.com/v1/forecast")           # no API key
def fetch(url: str, **params) -> dict:
    r = requests.get(url, params=params, timeout=10)  # give up on a stall
    r.raise_for_status()                              # 4xx/5xx -> exception
    return r.json()
def get_weather(city: str, units: str = "celsius", days: int = 0) -> dict:
    hits = fetch(GEO, name=city, count=1).get("results")
    if not hits:                                      # a result, not a crash
        return {"error": f"city not found: {city!r}"}
    w = fetch(API, latitude=hits[0]["latitude"],
              longitude=hits[0]["longitude"], temperature_unit=units,
              forecast_days=days + 1, current="temperature_2m",
              daily="temperature_2m_max,temperature_2m_min,weather_code")
    day = {k: v[days] for k, v in w["daily"].items()}  # the requested day
    if days == 0: day["temp"] = w["current"]["temperature_2m"]  # right now
    return {"city": city, "units": units, **day}
```

`get_weather("Hogwarts")` → `{'error': "city not found: 'Hogwarts'"}`

What `msg.tool_calls` actually contains

REQUEST · THE SCHEMA GOES WITH IT

```
messages = [{"role": "user", "content":
    "What's the weather in Berlin right now?"}]
resp = client.chat.completions.create(
    model=MODEL, messages=messages,
    tools=[WEATHER_TOOL])
msg = resp.choices[0].message
call = msg.tool_calls[0]
```

A list of requested actions. Each call has an ID, a function name, and arguments encoded as a **JSON string**.

ASSISTANT MESSAGE · RELEVANT FIELDS

```
{
  "role": "assistant",
  "content": "\n\n\n",
  "tool_calls": [{
    "id": "call_43b208ce8b3a4cb09651f03f",
    "type": "function",
    "function": {
      "name": "get_weather",
      "arguments": "{\"city\": \"Berlin\"}"
    }
  }]
}
```

Here `content` is whitespace; it can also be null. **The requested action is in `tool_calls`.** Live capture · 6 Nov 2026.

Return the result — then the model can answer

YOUR CODE EXECUTES AND REPLIES

```
import json
messages.append(msg)  # keep the assistant call
out = get_weather(
    **json.loads(call.function.arguments))
tool_reply = {
    "role": "tool",
    "tool_call_id": call.id,
    "content": json.dumps(out)}
messages.append(tool_reply)
```

`tool_call_id` is the same `call_43b208ce8b3a4cb09651f03f`. The serialized result includes `"temp": 3.9`.

NEXT REQUEST · ACCUMULATED MESSAGES

```
final = client.chat.completions.create(
    model=MODEL, messages=messages,
    tools=[WEATHER_TOOL]).choices[0].message
print(final.tool_calls)  # None in this run
```

Final answer, excerpt: “The current weather in Berlin is **3.9°C** with partly cloudy skies.”

```
if not final.tool_calls:
    return final.content  # inside run()
```

The ID matches the result to the request. No calls in the next turn → the loop returns the answer. Live capture · 6 Nov 2026.

Executing one call — failures become tool results

```python
TOOLS = {"get_weather": (get_weather, WeatherArgs)}   # name -> (fn, args)
def run_tool(call) -> dict:
    entry = TOOLS.get(call.function.name)
    if entry is None:                                  # hallucinated name
        out = {"error": f"unknown tool; available: {list(TOOLS)}"}
    else:
        fn, Args = entry
        try:                                           # validate, THEN call
            args = Args.model_validate_json(call.function.arguments)
            out = fn(**args.model_dump())
        except ValidationError as e:
            out = {"error": e.errors(include_url=False)}
        except requests.RequestException as e:         # network, timeout, 5xx
            out = {"error": f"weather service unavailable: {type(e).__name__}"}
    return {"role": "tool", "tool_call_id": call.id,
            "content": json.dumps(out, default=str)}
```

The arguments arrive as a **JSON string**. An unknown name, bad arguments, a dead network — each becomes a result the model can read, never an exception in your loop.

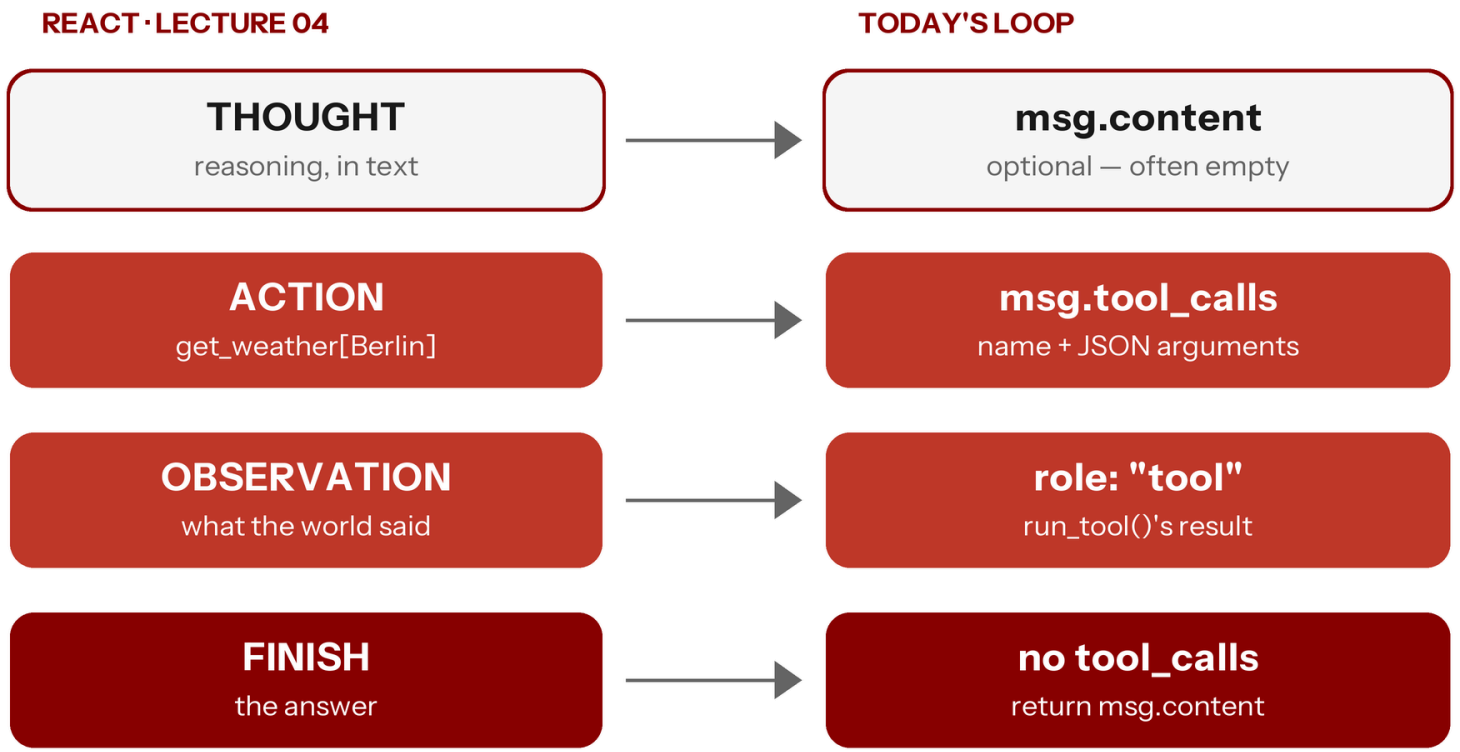
The loop — a cap, not `while True`

```
MAX_STEPS = 6
def run(task: str) -> str:
    messages = [{"role": "user", "content": task}]
    for step in range(MAX_STEPS):                # a cap, not `while True`
        msg = client.chat.completions.create(model=MODEL,
                  messages=messages, tools=[WEATHER_TOOL]
              ).choices[0].message
        messages.append(msg)
        if not msg.tool_calls:
            return msg.content                   # it answered: done
        for call in msg.tool_calls:
            print(step, call.function.name, call.function.arguments)
            messages.append(run_tool(call))
    raise RuntimeError(f"no answer after {MAX_STEPS} steps")
```

The same shape as last week's `run()` — bounded, and every call in a turn executed.

You have seen this loop: it is ReAct

No `tool_calls` in a turn is ReAct's **Finish**: the model answered, so `run()` returns. One honest difference: ReAct *requires* the Thought; function calling only *permits* it.



What the run printed

We sent	What came back
`get_weather` with `{"city": "Oslo", "days": 9}`	`{"error": [{"type": "less_than_equal", "loc": ["days"], "msg": "Input should be less than or equal to 3", …`
`get_forecast` — a tool that does not exist	`{"error": "unknown tool; available: ['get_weather']"}`
`{"city": "Oslo"}`, API unreachable	`{"error": "weather service unavailable: ConnectionError"}`
`run("What will the weather be in Berlin tomorrow, in fahrenheit?")`	step 0: `get_weather {"city": "Berlin", "days": 1, "units": "fahrenheit"}` → *Tomorrow in Berlin, the temperature will range from 48.3°F to 57.5°F, with light rain expected.*

The description doing its job: the model filled `days` and `units` from the schema, and read `weather_code: 61` as rain on its own.

Parallel calls in one turn

```
ask = "What's the weather right now in Berlin, Oslo and Madrid?"
resp = client.chat.completions.create(model=MODEL, tools=[WEATHER_TOOL],
           messages=[{"role": "user", "content": ask}])
calls = resp.choices[0].message.tool_calls
print(len(calls), "calls in one turn")
for c in calls:
    print(" ", c.id, c.function.name, c.function.arguments)
results = [run_tool(c) for c in calls]   # one tool message per call,
                                         # matched by tool_call_id
print([json.loads(r["content"])["temp"] for r in results])
```

```
3 calls in one turn
  call_c064… get_weather {"city": "Berlin"}
  call_bfa8… get_weather {"city": "Oslo"}
  call_3861… get_weather {"city": "Madrid"}
[11.3, 7.7, 13.5]
```

The **model's choice**, not a guarantee: nemotron `:free` models answered with one call per turn. The loop handles both. Only `city` was sent — the schema's defaults filled the rest.

DISCUSSION

You register twenty tools with your agent. When do those definitions cost you tokens?

Once, at setup? Or somewhere else? And what does your answer imply about giving an agent every tool it might conceivably need?

Every call, not once — measured

$$284 \text{ tokens} \times 30 \text{ turns} = 8{,}520 \text{ tokens}$$

One three-parameter tool on today's model: the same question costs 23 prompt tokens bare and 307 with `WEATHER_TOOL` attached.

THERE IS NO REGISTRATION STEP

Tool schemas are serialised into the prompt and re-sent on **every** call the loop makes — including the tools this task never touches. Twenty tools is a bill, and a distraction, on each request.

WHERE THIS ARGUMENT LEADS

It is why **skills** exist — you have already met them — and the case for **subagents** in Lecture 08. Both stop you carrying context you are not using.

Shopify hit the same wall — in behaviour, not tokens

0–20 TOOLS

"Clear boundaries, easy to debug, straightforward behavior"

20–50 TOOLS

"Boundaries become unclear, tool combinations start causing unexpected outcomes"

50+ TOOLS

"Multiple ways to accomplish the same task, system becomes difficult to reason about"

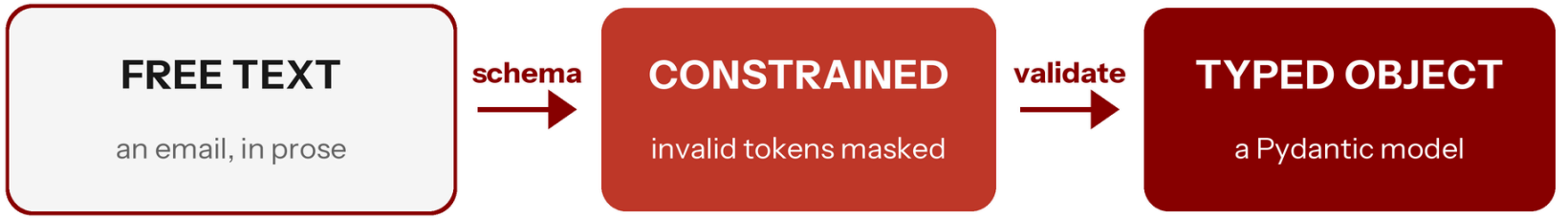
Shopify Sidekick's engineers, 2025: *"Resist the urge to add tools without clear boundaries."* Their fix returns instructions "alongside tool data exactly when they're needed" — the same economy as the skills you already know.

Structured Output

Data you can act on — and what to do when it fails

From prose to a typed object

The schema constrains **generation**, and validation confirms the result.




```
{ "recipient": "Dr. Anna Weber",
  "city": "Hamburg",
  "due": "2026-10-03" }
```

WHY CONSTRAIN AT DECODE
invalid output becomes impossible, not merely unlikely

the mechanism is Lecture 03; here it is the deliverable

Three ways to ask

PROMPT AND HOPE

"Reply in JSON." Works often, fails silently.

JSON MODE

Valid JSON (unless truncated) — but not *your* shape.

SCHEMA-CONSTRAINED

Your schema masks tokens at decode. Invalid JSON becomes impossible rather than unlikely.

The third is the constrained-decoding mechanism from Lecture 03, arriving as a feature you can use.

ONLY IF YOUR MODEL SUPPORTS IT

The course `.env`'s default free model does not list `structured_outputs` or `response_format` on OpenRouter. Today's runs used `dots-studio/dots-3-note-preview:free`, which lists both, plus `tools`. Free models come and go — check before class.

SKUF generation — prompt only vs JSON mode

```
PROMPT = """Generate one stereotypical fictional SKUF who hates leaving his apartment.
English only. Return ONLY JSON:
name: string; age: integer (18–100); occupation: string;
hobbies: list of strings; catchphrase: string;
grumpiness: integer (0–10); hours_outside_per_day: integer (0–24).
Use exactly these seven keys. Keep strings short and hobbies to two items."""
messages = [{"role": "user", "content": PROMPT}]
```

PROMPT ONLY · XING4.0

```
```json
{"name": "Uncle Bob", "age": 55,
 "occupation": "Freelance Writer",
 "hobbies": ["Junk food", "Board games"],
 "catchphrase": "Don't make me move!",
 "grumpiness": 8,
 "hours_outside_per_day": 0}
```
```

JSON MODE · DOTS3

```
response_format={"type": "json_object"}
```

```
{"name": "Barnaby", "age": 35,
 "occupation": "Software Developer",
 "hobbies": ["video games", "reading"],
 "catchphrase": "I'd rather be inside.",
 "grumpiness": 8,
 "hours_outside_per_day": 1}
```

Markdown fences break direct parsing; JSON mode enforces syntax, not schema.

The same SKUF request — now with a schema

REQUIRED FIELDS, TYPES AND RANGES

```python
from pydantic import BaseModel, ConfigDict, Field
class Skuf(BaseModel):
    model_config = ConfigDict(
        extra="forbid", strict=True)
    name: str
    age: int = Field(ge=18, le=100)
    occupation: str
    hobbies: list[str]
    catchphrase: str
    grumpiness: int = Field(ge=0, le=10)
    hours_outside_per_day: int = Field(ge=0, le=24)
```

SAME PROMPT · ENFORCED SCHEMA

```python
resp = client.chat.completions.parse(
    model=MODEL, messages=messages,
    response_format=Skuf)
msg = resp.choices[0].message
skuf = msg.parsed  # validated Skuf
```

Hours outside per day?

0·1·4·8

`extra="forbid"` rejects extra keys; `strict=True` rejects type coercion. The schema allows **any integer from 0 to 24** for outside hours.

Which character would you select?

1 · MARCUS

```
{
  "name": "Marcus",
  "age": 35,
  "occupation":
  "telemarketing clerk",
  "hobbies": [
    "feed the cat",
    "complain about weather"
  ],
  "catchphrase":
  "Why would I ever step outside?",
  "grumpiness": 9,
  "hours_outside_per_day": 0
}
```

2 · UNCLE BOB

```
{
  "name": "Uncle Bob",
  "age": 55,
  "occupation":
  "Freelance Writer",
  "hobbies": [
    "Junk food",
    "Board games"
  ],
  "catchphrase":
  "Don't make me move!",
  "grumpiness": 8,
  "hours_outside_per_day": 0
}
```

3 · BARNABY

```
{
  "name": "Barnaby",
  "age": 35,
  "occupation":
  "Software Developer",
  "hobbies": [
    "video games",
    "reading"
  ],
  "catchphrase":
  "I'd rather be inside.",
  "grumpiness": 8,
  "hours_outside_per_day": 1
}
```

From a validated SKUF to a database record

```
import sqlite3
from contextlib import closing
with closing(sqlite3.connect("skufs.sqlite3")) as db, db:
    db.execute("CREATE TABLE IF NOT EXISTS skufs "
               "(id INTEGER PRIMARY KEY, profile TEXT NOT NULL)")
    row = db.execute("INSERT INTO skufs(profile) VALUES (?)",
                     (skuf.model_dump_json(),))
    row_id = row.lastrowid                      # committed on leaving the block
with closing(sqlite3.connect("skufs.sqlite3")) as db:
    saved, = db.execute("SELECT profile FROM skufs WHERE id = ?",
                        (row_id,)).fetchone()
restored = Skuf.model_validate_json(saved)
print(row_id, restored.name, restored.hours_outside_per_day)
```

```
1 Marcus 0
```

SQLite is a local file; `?` binds the value. The **whole profile** is saved as JSON text. Your Python code writes the SQL and saves the record; the model supplies the data.

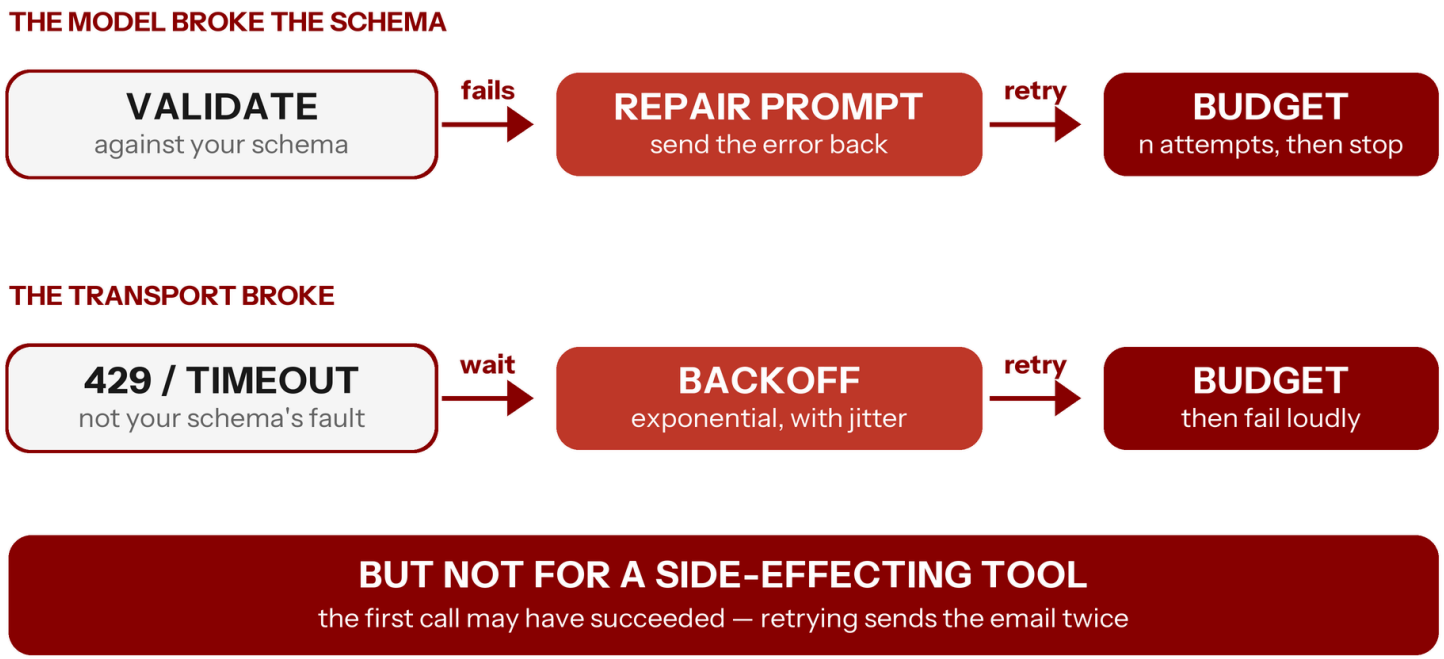
Pydantic as the schema

```
from datetime import date
from pydantic import BaseModel
class Shipment(BaseModel):
    recipient: str
    city: str
    due: date
EMAIL = ("Hi! Could you send the 40 sample kits to Dr. Anna Weber at our "
         "Hamburg lab? We need them by the 3rd of October 2026. "
         "Thanks, Tom")
resp = client.chat.completions.parse(model=MODEL,
    response_format=Shipment,        # the schema goes to the provider
    messages=[{"role": "user", "content": "Extract:\n" + EMAIL}])
shipment = resp.choices[0].message.parsed    # a Shipment, already validated
print(repr(shipment))
```

```
Shipment(recipient='Dr. Anna Weber', city='Hamburg', due=datetime.date(2026, 10, 3))
```

Two layers of failure

Typed output still fails — a schema violation, a truncated response, a refusal. And the transport fails underneath it. Same lesson, two levels.



retrying forever is not a strategy: decide what happens when the budget runs out

The repair loop — a forced violation

```
MAX_REPAIRS = 3
ASK = ("Return JSON with keys recipient, city, due. "
       "Write `due` in words, e.g. 'fifth of May'.\n")  # forced violation
def extract(email: str) -> Shipment:
    messages = [{"role": "user", "content": ASK + email}]
    for attempt in range(MAX_REPAIRS):
        raw = client.chat.completions.create(model=MODEL, messages=messages,
                    response_format={"type": "json_object"}
              ).choices[0].message.content
        try:
            return Shipment.model_validate_json(raw)
        except ValidationError as e:
            print(f"attempt {attempt}: {raw!r}")
            print("  ->", e.errors()[0]["msg"])
            messages += [{"role": "assistant", "content": raw},
                         {"role": "user", "content": f"Invalid: {e}"}]
    raise RuntimeError(f"no valid Shipment after {MAX_REPAIRS} attempts")
```

Forced by the prompt: JSON mode, no schema, the date asked for in words.

What the repair loop did

1. **Attempt 0** — JSON mode, no schema: `"due": "third of October 2026"`
2. **Pydantic rejects it** — `Input should be a valid date or datetime, invalid character in year`
3. **The error goes back** as the next user message — attempt 1 makes **the same mistake**
4. **Attempt 2 validates** — `Shipment(recipient='Dr. Anna Weber', city='Hamburg', due=datetime.date(2026, 10, 3))`

BUDGET THE ATTEMPTS

And decide what happens when they run out. This run used the last of three — one more failure and it is a `RuntimeError`, not an endless loop.

DISCUSSION

> ## Your `send_invoice` tool times out. Do you retry it?
>
> Show of hands — then justify it. What exactly does a timeout tell you about whether the invoice went out?

A timeout tells you nothing about what happened

1. **You only know you stopped waiting.** The invoice may have gone out and only the reply was lost — so a blind retry can send it twice, or charge a card twice.

2. **Make the call idempotent.** Your code attaches an **idempotency key**; the server stores the first result under it and answers any duplicate with that result. The action runs at most once — while the key is kept: Stripe may prune it after 24 h.

3. **Derive the key from the action, not from `tool_call_id`** — e.g. a hash of order id + amount. When the model re-issues the same send in a later turn, it arrives with a *new* call id — keyed on that, the retry goes through as a **second** invoice (we ran it: two sent).

4. **No key support downstream?** Check-then-act — look for the invoice before sending — or escalate to a human: Lecture 04's rule for irreversible actions.

Idempotency keys — the retry that runs once

```
import hashlib
sent, replies = [], {}                                  # the provider's side
def invoices_api(headers: dict, body: dict) -> dict:  # stub of POST /invoices
    key = headers["Idempotency-Key"]
    if key in replies:                                  # seen: replay, do not send
        return replies[key] | {"replayed": True}
    sent.append(body)                                   # the side effect, once
    replies[key] = {"invoice": f"inv_{len(sent)}", "replayed": False}
    return replies[key]
def send_invoice(order_id: str, cents: int) -> dict:  # the tool the model calls
    key = hashlib.sha256(f"invoice:{order_id}:{cents}".encode()).hexdigest()
    return invoices_api({"Idempotency-Key": key},     # from the ACTION,
                        {"order": order_id, "cents": cents})  # not tool_call_id
print("turn 3:", send_invoice("ord_42", 1999))  # sent; the reply timed out
print("turn 5:", send_invoice("ord_42", 1999))  # model re-issues: new call id
print("invoices sent:", len(sent))
```

```
turn 3: {'invoice': 'inv_1', 'replayed': False}
turn 5: {'invoice': 'inv_1', 'replayed': True}
invoices sent: 1
```

Rate limits — backoff with jitter

```python
import random, time
from openai import RateLimitError
def create_with_backoff(max_retries=5, base=0.5, cap=20.0, **kw):
    c = client.with_options(max_retries=0)  # the SDK retries 2x itself
    for attempt in range(max_retries):
        try:
            return c.chat.completions.create(**kw)
        except RateLimitError:
            ceiling = min(cap, base * 2 ** attempt)     # exponential
            delay = random.uniform(0, ceiling)          # full jitter
            print(f"429 on attempt {attempt}; sleeping {delay:.2f}s")
            time.sleep(delay)
    raise RuntimeError(f"still rate-limited after {max_retries} tries")
# Never retry a side-effecting tool this way: a call that "failed"
# may already have succeeded.
```

`with_options(max_retries=0)` is load-bearing: the SDK already retries a 429 twice. One retry layer, not two.

Against a real 429

RECOVERED · 4 TRIES ALLOWED

```
429 on attempt 0; sleeping 0.89s
429 on attempt 1; sleeping 0.79s
-> ready
```

BUDGET EXHAUSTED · 6 TRIES ALLOWED

```
429 on attempt 0; sleeping 1.20s
429 on attempt 1; sleeping 3.97s
429 on attempt 2; sleeping 8.00s
…
429 on attempt 5; sleeping 7.94s
RuntimeError: still rate-limited after 6 tries
```

Live, on 2026-09-23: a free OpenRouter model that was genuinely rate-limited upstream, `base=2.0`.

A FAILURE THIS HANDLER DOES NOT CATCH

One free-tier call came back as **HTTP 200** with `choices: null` and an upstream 503 in the body. The SDK does not raise, so `r.choices[0]` fails with `TypeError`. A real client checks `r.choices` first.

DISCUSSION

A pizza chain's order bot · a bank's transfer assistant · an internal analytics Q&A bot.

For each: which tools have side effects? Which need an idempotency key, a human confirmation, structured output — and where does each break first?

Three bots — the taxonomy decides the handling

PIZZA ORDER BOT

Side effect: `place_order`. **Needs:** a key from the cart, the order as a schema, a read-back. **Breaks:** a retry after a timeout → two pizzas; a stale menu enum fails every order.

BANK TRANSFER ASSISTANT

Side effect: `transfer`, irreversible. **Needs all three:** a validated schema, a human confirmation, a key per confirmed request. **Breaks:** a key from (payee, amount) blocks a genuine second transfer.

ANALYTICS Q&A BOT

Side effects: none — if the database role says so. **Needs:** read-only credentials; the SQL it ran. Stripe splits `stripe_api_read` / `_write`. **Breaks:** "read-only" only in the description.

Retrieval: retry · side effect: a key · irreversible: a key **and** a human (Stripe confirms refunds and outbound payments) · leaves for an API: a schema.

Instrumentation

You cannot debug what you did not record

Log every call — one JSON record, always

```
RUN_ID = uuid.uuid4().hex[:8]                    # ties one run's records
def logged_create(step: int, **kw):
    t0, resp, err = time.perf_counter(), None, None
    try:
        resp = client.chat.completions.create(**kw)
        return resp
    except Exception as e:
        err = type(e).__name__
        raise
    finally:                                     # one record per call, always
        u = resp.usage if resp else None
        log.info(json.dumps({"run_id": RUN_ID, "step": step,
            "kind": "model", "model": kw["model"],
            "prompt_tokens": u and u.prompt_tokens,
            "completion_tokens": u and u.completion_tokens,
            "latency_ms": round((time.perf_counter() - t0) * 1000),
            "error": err}))
```

What the log recorded

run_id	step	model	prompt	completion	latency_ms	error
9953729b	0	dots-studio/dots-3-note-preview:free	301	78	3841	null
9953729b	1	no-such/model	null	null	53	BadRequestError

THE FAILED CALL STILL HAS A RECORD

The `finally` is the point. Tool calls get the same wrapper with `"kind": "tool"` — and the `run_id` turns the lines into one trajectory, which Lecture 13 teaches you to read.

FREE TIER, MEASURED

4 s on today's `:free` model; 40–104 s on another one a fortnight ago. The latency field is how you find that out.

What a run costs

```
from collections import Counter
usage = Counter()                       # one per run
def create(**kw):                       # every call goes through this
    resp = client.chat.completions.create(**kw)
    usage["calls"] += 1
    usage["prompt_tokens"] += resp.usage.prompt_tokens
    usage["completion_tokens"] += resp.usage.completion_tokens
    return resp
ask = [{"role": "user",
        "content": "Is it warmer in Berlin or Oslo right now?"}]
bare = create(model=MODEL, messages=ask)
tooled = create(model=MODEL, messages=ask, tools=[WEATHER_TOOL])
print("schema adds", tooled.usage.prompt_tokens - bare.usage.prompt_tokens,
      "prompt tokens to every call")
print(dict(usage))
```

```
schema adds 284 prompt tokens to every call
{'calls': 2, 'prompt_tokens': 330, 'completion_tokens': 518}
```

On `:free` the dollar cost is zero — multiply tokens by your own model's price.

Tools that return images

A TOOL WHOSE RESULT IS A PICTURE

A tool returns an image, and the model reads it as part of the tool result. The minimum multimodal primitive.

WHY IT MATTERS

Document retrieval in practice is rarely text-only. Vision model internals and OCR pipelines are out of scope — the primitive is all you need.

Now put this tool in a loop.

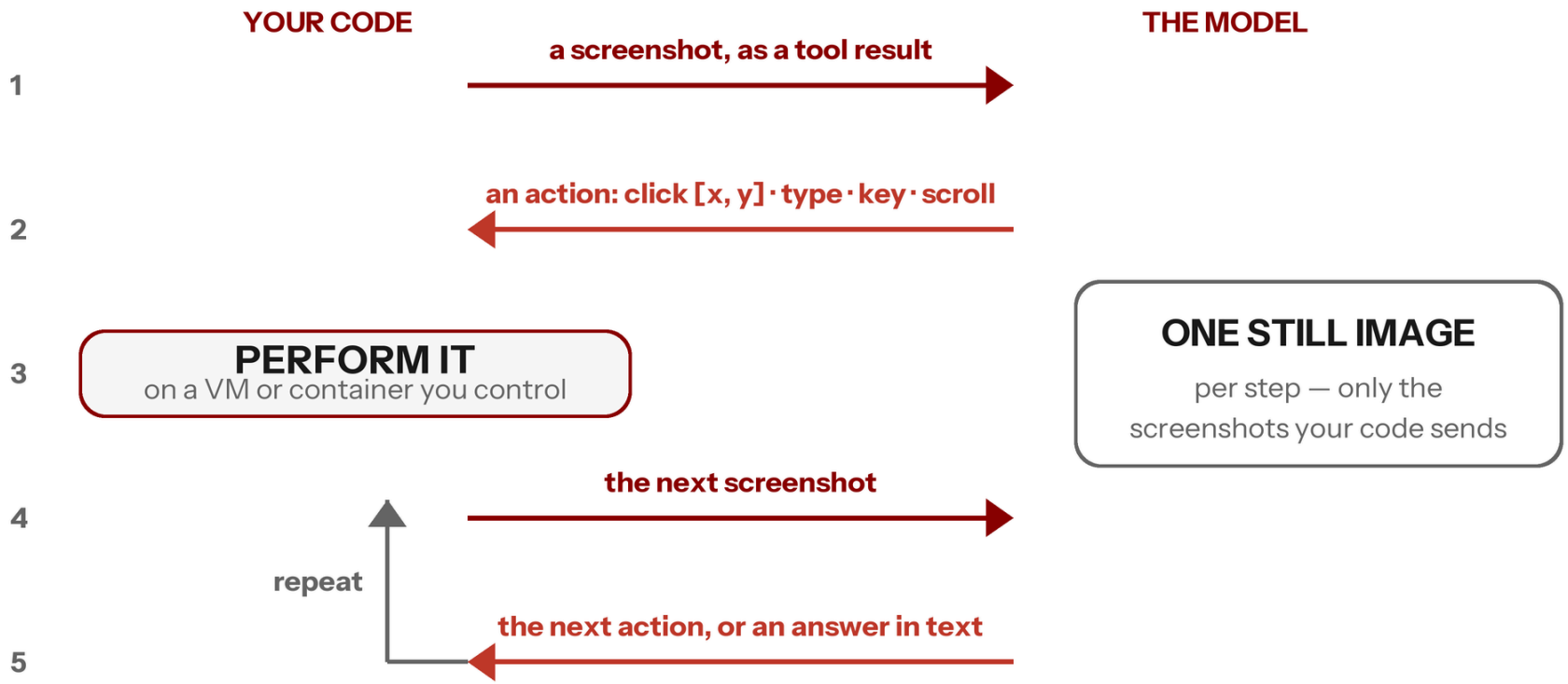
That is the whole of the next block.

Computer Use

And browser agents

The image tool, run in a loop

Your code acts, on a machine you control — **the model never touches anything**.



the image tool from the previous block, run in a loop — nothing else is new

One built-in tool, not a schema you write

```
# anthropic==1.8.0 · computer_toolset_20260801 · docs checked 2026-09-23
# UNVERIFIED against the live API — SDK request/response shape checked offline
import anthropic
client = anthropic.Anthropic()                      # ANTHROPIC_API_KEY from the env
COMPUTER = {"type": "computer_toolset_20260801"}  # no name, no screen size, no beta header
messages = [{"role": "user",
             "content": "Open the course page and find next week's lecture."}]
```

WHY A SECOND SDK

Computer use is a provider-native **built-in tool type**, not a function schema — one `tools` entry, no JSON Schema. So it is shown on the vendor's own SDK.

OPENAI'S EQUIVALENT

The Responses API tool `{"type": "computer"}`, returning a `computer_call` with an `actions` array. Its guide now steers its newest model to code execution instead.

The loop — you have written this before

```python
for step in range(30):                                  # a cap, never `while True`
    resp = client.messages.create(model="claude-opus-5", max_tokens=1024,
                                  tools=[COMPUTER], messages=messages)
    messages.append({"role": "assistant", "content": resp.content})
    calls = [b for b in resp.content if b.type == "tool_use"]
    if not calls:
        break                                           # the model answered in text
    results = []
    for c in calls:                                     # several per turn: run them in order
        out = desktop.do(c.name, c.input)               # YOUR code: screenshot, click x,y, type
        results.append({"type": "tool_result", "tool_use_id": c.id,
                        "toolset_name": "computer", "content": out})
    messages.append({"role": "user", "content": results})
print(resp.content[0].text)
```

One turn can carry several actions — a click, then a screenshot: run them in order, one result each, each echoing `toolset_name`. `desktop` is your VM.

Pixels or structure?

	Pixels — screenshot + coordinates	DOM / accessibility tree
Works on	any application, incl. remote desktop	web pages only
Model sees	an image, every step	a text list of elements, each with a reference
It acts by	clicking at `[x, y]`	naming an element: `[ref_2]`, index `33`
Cost	**2,691** image tokens per 1920×1080 screenshot on Claude 4.7+, every step	text only; no vision model needed
Breaks when	layout shifts — coordinates go stale	no useful node: canvas, video, cross-origin iframes

Oversized screenshots are **rejected**, not downscaled: resize them yourself.

Two designs on the structure side

PLAYWRIGHT MCP · 0.0.82

An **MCP server** — a standard way to plug a ready-made tool server into an agent (Lecture 11). It "uses Playwright's accessibility tree, not pixel-based input": `browser_click` takes an element reference, and its own screenshot tool says you can't act on a screenshot. Coordinates only behind opt-in `--caps vision`.

BROWSER-USE · 0.13.10

A Python library that serialises the page into indexed elements; its click takes an element **index**. By default it also sends a screenshot with the elements boxed, and accepts coordinates as a fallback — a **hybrid**.

Named as designs, not taught as APIs — frameworks are Lecture 09's.

Not a binary

> **Act on structure where the page has it; fall back to pixels where it does not.**
>
> Anthropic's browser toolset: element references survive "layout shifts and reflows that make pixel coordinates fragile"; canvas, video and cross-origin iframes fall back to screenshots. Pick pixels when you must leave the browser.

HOW WELL DO THESE AGENTS WORK?

Lecture 12 — OSWorld, WebArena Verified.

WHAT CAN A WEB PAGE DO TO YOUR AGENT?

"Instructions on webpages or contained in images might override your instructions" — indirect injection, Lecture 13.

Summary

COVERED

- Raw SDK, pinned; the key from `.env`; a real weather API
- Tools: name, description, schema, taxonomy
- The loop made correct: typed, validated, capped, parallel
- The loop is ReAct: no tool call means Finish
- Structured output, repair, backoff
- Logging and cost; computer use as a loop
- Three bots: a key, a human or a schema?

KEEP THESE

- The model emits text; your code acts
- The model sees a tool's name, description and schema — nothing else
- Never retry a side effect blindly
- No `run_id`, no debugging
- One still image per step, and only the ones you send
- A turn with no tool call is the agent saying "done"

References · tools, output and retries

- OpenAI, Anthropic · **Tool use / function calling** · developers.openai.com/api/docs/guides/function-calling
- OpenAI · **Structured model outputs** · developers.openai.com/api/docs/guides/structured-outputs · checked 6 Nov 2026
- Python 3.12 · **sqlite3 — DB-API 2.0 interface** · docs.python.org/3.12/library/sqlite3.html
- Pydantic · **Model validation** · pydantic 2.13.5
- Willison · **I think "agent" may finally have a widely enough agreed upon definition to be useful jargon now** · simonwillison.net/2025/Sep/18/agents/ · 18 Sep 2025
- Open-Meteo · **Weather Forecast and Geocoding APIs** · open-meteo.com/en/docs (free for non-commercial use, no key; checked 2026-10-08)
- Constrained decoding · see Lecture 03's sources
- Featonby · **Making retries safe with idempotent APIs** · AWS Builders' Library · aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/
- Stripe · **Idempotent requests** · docs.stripe.com/api/idempotent_requests (checked 2026-09-23)
- Jena, Dalal · **The Idempotency-Key HTTP Header Field** · IETF httpapi WG · draft-ietf-httpapi-idempotency-key-header-07 (Oct 2025; expired Apr 2026, not an RFC)

References · computer use and deployments

- Anthropic · **Computer use tool** · platform.claude.com/docs/en/agents-and-tools/tool-use/computer-use-tool (checked 2026-09-23)
- Anthropic · **Browser use tool** · platform.claude.com/docs/en/agents-and-tools/tool-use/browser-use-tool (checked 2026-09-23)
- Anthropic · **Vision — resolution and token cost** · platform.claude.com/docs/en/build-with-claude/vision
- OpenAI · **Computer use** · developers.openai.com/api/docs/guides/tools-computer-use (checked 2026-09-23)
- Microsoft · **Playwright MCP** · github.com/microsoft/playwright-mcp · @playwright/mcp 0.0.82
- browser-use · **browser-use** · github.com/browser-use/browser-use · 0.13.10
- Klarna · **Press release — AI assistant handles two-thirds of customer service chats in its first month** · 27 Feb 2024
- McNamara · **Building production-ready agentic systems: Lessons from Shopify Sidekick** · Shopify Engineering · 2025
- Stripe · **Model Context Protocol (MCP)** · docs.stripe.com/mcp (checked 2026-09-28)

What's Next · Lecture 06: RAG: Basics, Practice, and Custom Retrieval

WHAT'S COVERED

- Chunking, embeddings for retrieval, vector stores, hybrid search
- A pipeline built live with LlamaIndex
- Why the library is not enough: reranking, hard negatives, retrieval metrics

HOW TODAY CONNECTS

- The demo assumes you can run and debug SDK code — today's
- Its retrieval is a fixed pipeline; agent-driven retrieval is Lecture 07
- That pipeline is what Lecture 13 attacks

Understand retrieval → build it with a library → understand why the library is not enough.

The third movement is the reason the lecture exists in this form.

Thank You

Questions & Discussion

Lecture 05 · LLM Agent Development